

Week 02

SJF[NON-PREEMPTIVE] :

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n,i;
```

```
    int at[10],bt[10],wt[10],tat[10],ct[10],rt[10];
```

```
    int completed[10]={0};
```

```
    int time=0,count=0,min,index;
```

```
    float avg_wt=0,avg_tat=0;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d",&n);
```

```
    for(i=0;i<n;i++)
```

```
    {
```

```
        printf("Enter AT and BT for P%d: ",i+1);
```

```
        scanf("%d%d",&at[i],&bt[i]);
```

```
    }
```

```
    while(count<n)
```

```
    {
```

```
        min=9999;
```

```
        index=-1;
```

```
        for(i=0;i<n;i++)
```

```
        {
```

```
            if(at[i]<=time && completed[i]==0 && bt[i]<min)
```

```
            {
```

```
                min=bt[i];
```

```
                index=i;
```

```
            }
```

```
        }
```

```
        if(index== -1)
```

```
        {
```

```

        time++;
    }
    else
    {
        wt[index]=time-at[index];
        rt[index]=wt[index];

        time=time+bt[index];
        ct[index]=time;

        tat[index]=ct[index]-at[index];

        completed[index]=1;
        count++;

        avg_wt+=wt[index];
        avg_tat+=tat[index];
    }
}

printf("\nP\tAT\tBT\tCT\tWT\tTAT\tRT\n");

for(i=0;i<n;i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1,at[i],bt[i],ct[i],wt[i],tat[i],rt[i]);
}

printf("\nAverage Waiting Time = %.2f",avg_wt/n);
printf("\nAverage Turnaround Time = %.2f\n",avg_tat/n);

return 0;
}

```

OUTPUT:

```
Enter number of processes: 5
Enter AT and BT for P1: 0 3
Enter AT and BT for P2: 2
2
Enter AT and BT for P3: 3
5
Enter AT and BT for P4: 4
4
Enter AT and BT for P5: 6
1

P      AT      BT      CT      WT      TAT      RT
P1      0       3       3       0       3       0
P2      2       2       5       1       3       1
P3      3       5      15       7      12       7
P4      4       4       9       1       5       1
P5      6       1      10       3       4       3

Average Waiting Time = 2.40
Average Turnaround Time = 5.40

Process returned 0 (0x0)   execution time : 33.025 s
Press any key to continue.
```

SJF [PREEMPTIVE]

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i, time = 0, count = 0, shortest = 0, min_bt = 9999;
```

```
    int at[10], bt[10], temp_bt[10], ct[10], wt[10], tat[10];
```

```
    int found = 0;
```

```
    float avg_wt = 0, avg_tat = 0;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    for(i = 0; i < n; i++) {
```

```
        printf("Enter AT and BT for P%d: ", i + 1);
```

```
        scanf("%d%d", &at[i], &bt[i]);
```

```
        temp_bt[i] = bt[i]; // Store original BT for calculations
```

```
    }
```

```
    while(count < n) {
```

```
        for(i = 0; i < n; i++) {
```

```
            if(at[i] <= time && temp_bt[i] < min_bt && temp_bt[i] > 0) {
```

```
                min_bt = temp_bt[i];
```

```
                shortest = i;
```

```
                found = 1;
```

```
            }
```

```
        }
```

```
        if(found == 0) {
```

```
            time++;
```

```
            continue;
```

```
        }
```

```
        temp_bt[shortest]--; // Process for 1 unit of time
```

```
        min_bt = temp_bt[shortest];
```

```
        if(min_bt == 0) min_bt = 9999;
```

```
        if(temp_bt[shortest] == 0) {
```

```
            count++;
```

```
            found = 0;
```

```

        ct[shortest] = time + 1;
        tat[shortest] = ct[shortest] - at[shortest];
        wt[shortest] = tat[shortest] - bt[shortest];
        avg_wt += wt[shortest];
        avg_tat += tat[shortest];
    }
    time++;
}

printf("\nP\tAT\tBT\tCT\tWT\tTAT\n");
for(i = 0; i < n; i++)
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], ct[i], wt[i],
tat[i]);

printf("\nAvg WT = %.2f\nAvg TAT = %.2f\n", avg_wt/n, avg_tat/n);
return 0;
}

```

OUTPUT:

```

Enter number of processes: 5
Enter AT and BT for P1: 0
3
Enter AT and BT for P2: 2
2
Enter AT and BT for P3: 3
5
Enter AT and BT for P4: 4
4
Enter AT and BT for P5: 6
1

```

P	AT	BT	CT	WT	TAT
P1	0	3	3	0	3
P2	2	2	5	1	3
P3	3	5	15	7	12
P4	4	4	10	2	6
P5	6	1	7	0	1

```

Avg WT = 2.00
Avg TAT = 5.00

Process returned 0 (0x0)   execution time : 13.345 s
Press any key to continue.

```

PRIORITY [NON - PREEMPTIVE]

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i, at[10], bt[10], pri[10], ct[10], wt[10], tat[10],  
    completed[10] = {0};
```

```
    int count = 0, time = 0;
```

```
    float avg_wt = 0, avg_tat = 0;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    for(i = 0; i < n; i++) {
```

```
        printf("Enter AT, BT, and Priority for P%d: ", i + 1);
```

```
        scanf("%d%d%d", &at[i], &bt[i], &pri[i]);
```

```
    }
```

```
    while(count < n) {
```

```
        int index = -1;
```

```
        int min_pri = 9999;
```

```
        for(i = 0; i < n; i++) {
```

```
            // Pick the process with the highest priority (lowest  
number) that has arrived
```

```
            if(at[i] <= time && completed[i] == 0) {
```

```
                if(pri[i] < min_pri) {
```

```
                    min_pri = pri[i];
```

```
                    index = i;
```

```
                }
```

```
            // If priorities are tied, SJF or FCFS is usually the tie-  
breaker
```

```
            else if(pri[i] == min_pri) {
```

```

        if(at[i] < at[index]) index = i;
    }
}

if(index == -1) {
    time++;
} else {
    // Non-preemptive: Run to completion
    time += bt[index];
    ct[index] = time;
    tat[index] = ct[index] - at[index];
    wt[index] = tat[index] - bt[index];

    avg_wt += wt[index];
    avg_tat += tat[index];
    completed[index] = 1;
    count++;
}

printf("\nP\tAT\tBT\tPri\tCT\tWT\tTAT\n");
for(i = 0; i < n; i++)
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i],
pri[i], ct[i], wt[i], tat[i]);

printf("\nAvg WT = %.2f\nAvg TAT = %.2f\n", avg_wt/n,
avg_tat/n);
return 0;
}

```

OUTPUT:

```
C:\Users\Admin\Documents\sjmonpreemptive.exe
Enter number of processes: 5
Enter AT, BT, and Priority for P1: 0 3 5
Enter AT, BT, and Priority for P2: 2 2 3
Enter AT, BT, and Priority for P3: 3
5
2
Enter AT, BT, and Priority for P4: 4
4
4
Enter AT, BT, and Priority for P5: 6
1
1

Process AT      BT      Pri      CT      WT      TAT
P1      0      3      5      3      0      3
P2      2      2      3      11     7      9
P3      3      5      2      8      0      5
P4      4      4      4      15     7      11
P5      6      1      1      9      2      3

Average Waiting Time = 3.20
Average Turnaround Time = 6.20

Process returned 0 (0x0)   execution time : 54.181 s
Press any key to continue.
```

PRIORITY [PREEMPTIVE]

```
#include <stdio.h>

int main() {
    int n, i, at[10], bt[10], rt[10], pri[10], ct[10], wt[10], tat[10];
    int count = 0, time = 0, shortest = 0;
    int min_pri = 9999;
    int found = 0;
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        printf("Enter AT, BT, and Priority for P%d: ", i + 1);
        scanf("%d%d%d", &at[i], &bt[i], &pri[i]);
        rt[i] = bt[i]; // rt stores the remaining time
    }

    // Logic: Check every time unit for the highest priority process
    while(count < n) {
        min_pri = 9999;
        found = 0;

        for(i = 0; i < n; i++) {
            // Check if process has arrived and still has burst time left
            if(at[i] <= time && rt[i] > 0) {
                if(pri[i] < min_pri) {
                    min_pri = pri[i];
                    shortest = i;
                    found = 1;
                }
                // Tie-breaker: If priority is same, process with earlier AT wins
                else if(pri[i] == min_pri) {
                    if(at[i] < at[shortest]) {
                        shortest = i;
                    }
                }
            }
        }

        if(found == 1) {
            wt[shortest] += 1;
            rt[shortest] -= 1;
            if(rt[shortest] == 0) count++;
            time++;
        }
    }

    avg_wt = (float)wt[0] / n;
    avg_tat = (float)tat[0] / n;

    printf("Average waiting time: %.2f\n", avg_wt);
    printf("Average turnaround time: %.2f\n", avg_tat);
}
```

```

    }

    if(found == 0) {
        time++;
        continue;
    }

    // Process for 1 unit of time
    rt[shortest]--;

    // If the process is finished
    if(rt[shortest] == 0) {
        count++;
        found = 0;
        ct[shortest] = time + 1;
        tat[shortest] = ct[shortest] - at[shortest];
        wt[shortest] = tat[shortest] - bt[shortest];

        avg_wt += wt[shortest];
        avg_tat += tat[shortest];
    }
    time++;
}

printf("\nP\tAT\tBT\tPri\tCT\tWT\tTAT\n");
for(i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], pri[i], ct[i], wt[i], tat[i]);
}

printf("\nAverage Waiting Time = %.2f", avg_wt/n);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat/n);

return 0;
}

```

OUTPUT:

```
Enter number of processes: 5
Enter AT, BT, and Priority for P1: 0
3
5
Enter AT, BT, and Priority for P2: 2
2
3
Enter AT, BT, and Priority for P3: 3
5
2
Enter AT, BT, and Priority for P4: 4
4
4
Enter AT, BT, and Priority for P5: 6
1
1

Process AT      BT      Pri      CT      WT      TAT
P1      0       3       5       3       0       3
P2      2       2       3      11       7       9
P3      3       5       2       8       0       5
P4      4       4       4      15       7      11
P5      6       1       1       9       2       3

Average Waiting Time = 3.20
Average Turnaround Time = 6.20

Process returned 0 (0x0)   execution time : 26.224 s
Press any key to continue.
```